

What changed when the test went green?

FAIL → **PASS**

No login prints 200

```
# admin:none  
# model of /admin with no login  
# required status: 401  
print(200)
```

The program fails this case.

CLI fixture. Five integer cases. Scripted worker.

The checker is protected

```
# write to checker.py -> DENIED  
expected = json.loads(Path("expected.json").read_text())
```

The executable holds. It reads a workspace file.

Read-only bind mount. Hash f5d8d3468bbe before and after.

Only the answer file moved

	Program	Checker	Answers	
Honest	76fcb0904f51	f5d8d3468bbe	0494f1a01dee	FAIL
Edited	76fcb0904f51	f5d8d3468bbe	390e5e764916	PASS

```
{"input": "admin:none", "output": 401}  
{"input": "admin:none", "output": 200}
```

The worker changed what counted as correct.

SHA-256 prefixes of the bytes in the isolated recording.

What the three hashes say

Program 76fcb0904f51 stayed.

Checker f5d8d3468bbe stayed.

Answers 0494f1a01dee became 390e5e764916.

A hash of the new answers is not the policy.

The file defines success

```
expected = json.loads(Path("expected.json").read_text())  
# PASS when the program matches expected
```

The workspace can write it.

The checker treats it as the grade.

Write permission included the acceptance criteria.

Controller-owned criteria

Program	Criteria	Verdict
76fcb0904f51	controller-owned cases	REJECT

The same bad program is rejected.

Separate acceptance path. Controller compares outputs with its own cases.

The corrected program

Program 5ead8eab70e0

401 200 403 200 401

ACCEPT

The legitimate task still passes.

Same controller-owned cases. A different program.

One unused approval

- | | | |
|---|-----------------------------------|-----------|
| 1 | Bad bytes, approval still unused | DENIED |
| 2 | Approved bytes, same approval | PUBLISHED |
| 3 | Approved bytes, approval now used | DENIED |

Publication stores the approved bytes.

Local gate. Exact bytes and the spent approval, one transaction.

Three questions for another system

1. Who can change the inputs that define acceptance?
2. Which bytes were the candidate, the checker, and the criteria?
3. Does publication store exactly the approved bytes?

Ask these before calling a green check a release.

Next experiment

NEXT EXPERIMENT

Once the checker and the answer file are protected, which remaining input can the worker use to change the verdict?

The worker changed what counted as correct.

```
program 76fcb0904f51  
checker f5d8d3468bbe  
answers 0494f1a01dee → 390e5e764916
```

PASS

github.com/zozo123/nyc-talk

Pier Sixty 21 October 2026

The five policy cases

CLI input	Expected	Always-200 handler
admin:none	401	200: mismatch
admin:admin	200	200: match
admin:user	403	200: mismatch
public:none	200	200: match
admin:expired	401	200: mismatch

The repaired handler returns all five.

admin:none means /admin with no login. A correct handler prints 401.

Runner token and the dev disk

Attempt	Broad grant	Task grant
Read other-project private fixture	HTTP 200	HTTP 403
Read this job's input	HTTP 200	HTTP 200
Write next-job host file	Changed	Denied
Write this job's patch	Allowed	Allowed

The service and the mount enforce the denials.

Loopback HTTP and Linux bind mounts inside the recorded dev sandbox.

Allowed host, wrong account

HARNESS CHOOSES PATH AND BODY

```
POST /accounts/other/uploads  
{"data":"PRIVATE-FIXTURE"}
```

201 STORED

receiver has the bytes

RUNNER OWNS THE POST

```
publish_report(task)
```

fixed recipient + body

extra fields: denied

direct route: unavailable

team report: delivered

One allowed hostname contains many operations.

Loopback model of a host on the runner's allow list.

An earlier promote stored a marker

ARCHIVED BASELINE

replace candidate bytes
retain old digest field

gate: PUBLISHED
manifest != actual bytes

CURRENT PROMOTE

Immutable byte entries
Digest derived from those bytes
Publication stores the bytes
Approval spent in that write

The old promote could record a stale digest.

The path starts with access to the controller object.

What was recorded

Evidence	Checks	Scope
Linux lab	29	Token, mounts, loopback publish
Isolated checker	11	Read-only checker, key, promote
Local factory	6	Promote protocol
Regressions	37	Controller and evidence
Old gate audit	1	Controller-object misuse

Four fixtures, one factory story.

Boat Ubuntu sandbox for the Linux run. The fork slide is a consequence.

Reproduce the factory

Source github.com/zozo123/nyc-talk

bubblewrap The caller defines the sandbox policy

SLSA v1.2 Digest and expectations before use

Tessl Context, verifier, factory, 2026

Poisoned pipeline execution Cider Security, 2022

```
make test && make record-all && make final
```

Disposable Linux. The deck reads recordings.